

✓ Linear Regression

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy.stats import linregress
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
```

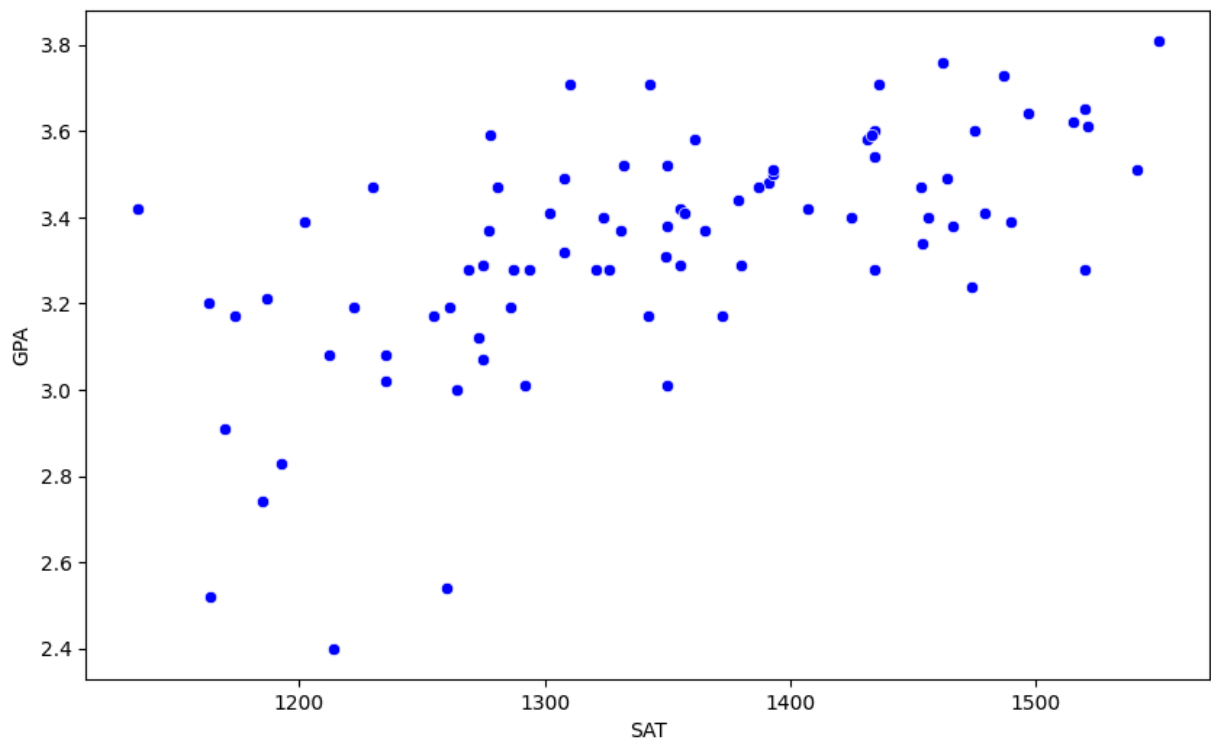
```
df = pd.read_csv('RegData.csv')
```

```
df.head()
```

	StID	SAT	GPA
0	101	1355	3.42
1	102	1391	3.48
2	103	1170	2.91
3	104	1357	3.41
4	105	1326	3.28

```
X = df["SAT"]
Y = df["GPA"]
```

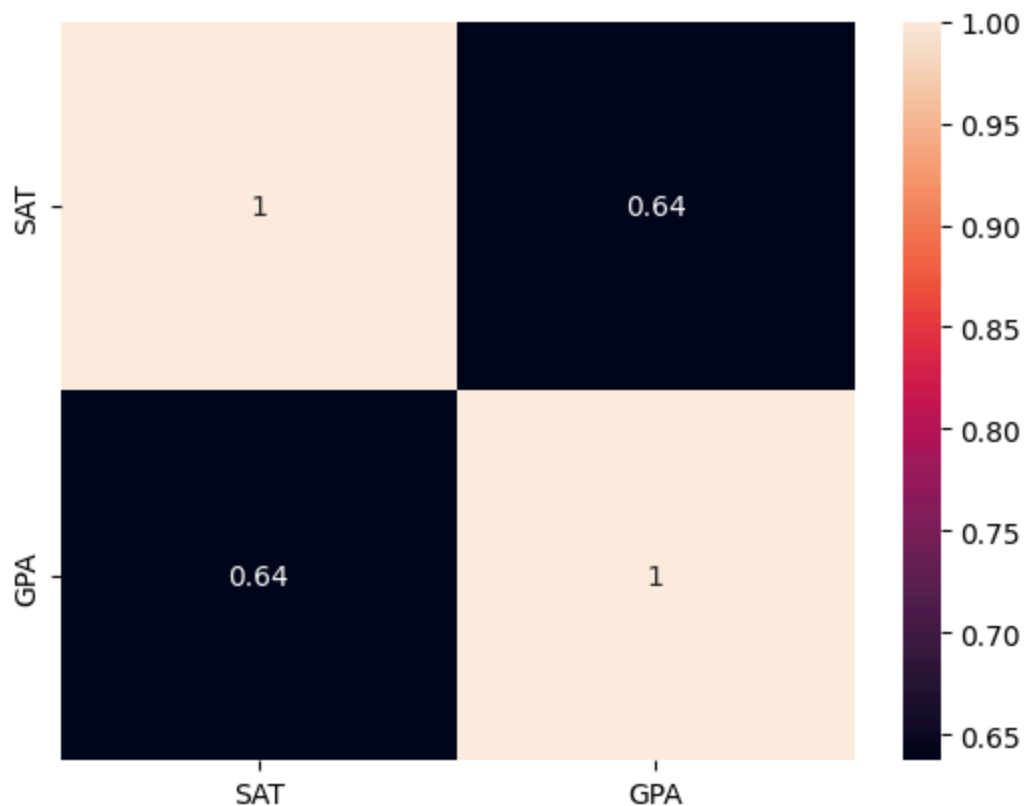
```
plt.figure(figsize=(10,6))
sns.scatterplot(x=X, y=Y, color="blue")
plt.xlabel("SAT")
plt.ylabel("GPA")
plt.show()
```



```
corr = df[["SAT", "GPA"]].corr()
```

```
sns.heatmap(corr, annot=True)
```

<Axes: >

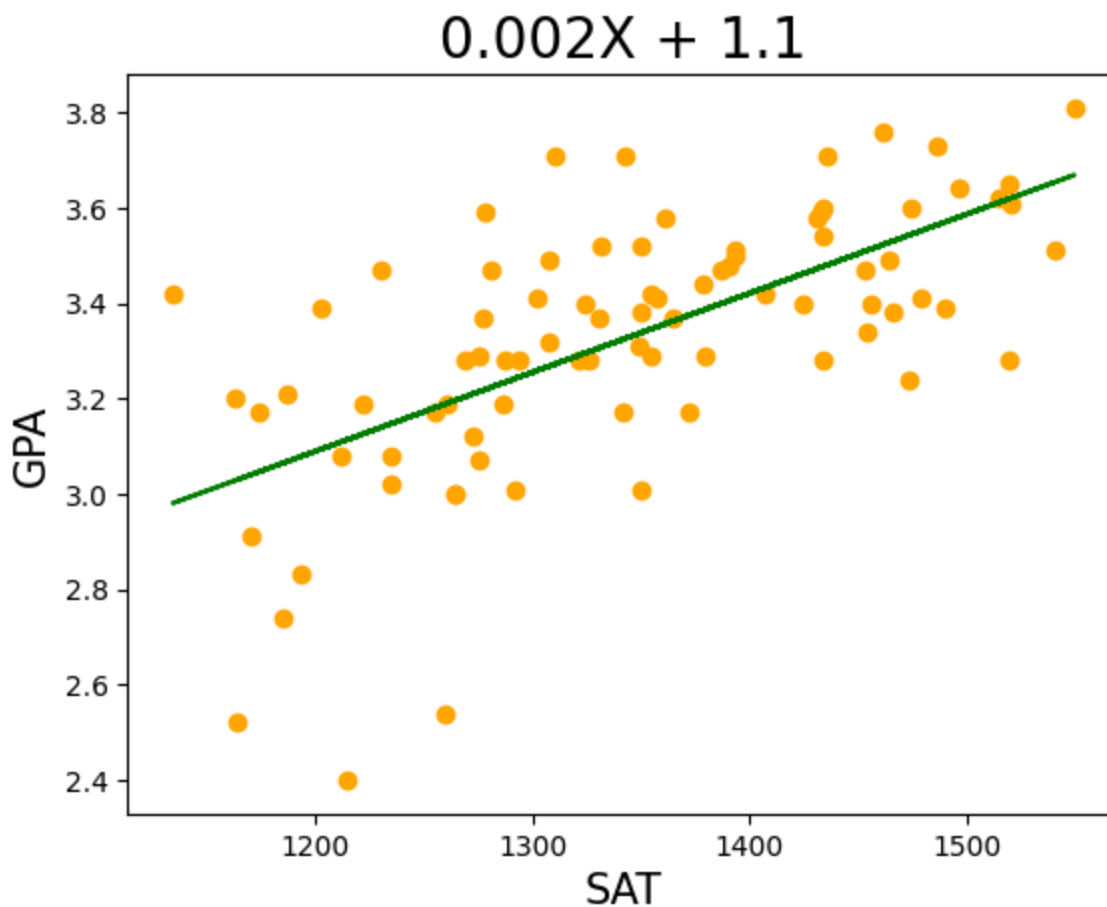


```
slope, intercept, r_value, p_value, std_err = linregress(X,Y)
```

```
print(f"Slope : {slope}")
print(f"Intercept : {intercept}")
print(f"Y = {round(slope, 3)}X + {round(intercept, 2)}")
```

```
Slope : 0.001655688050092813
Intercept : 1.1028843247066882
Y = 0.002X + 1.1
```

```
y1 = slope*X + intercept
plt.scatter(X, Y, color="orange")
plt.plot(X, y1, color="green")
plt.xlabel("SAT", fontsize=15)
plt.ylabel("GPA", fontsize=15)
plt.title(f"{round(slope, 3)}X + {round(intercept, 2)}", fontsize=20)
plt.show()
```



✓ Prediction

```
givenSAT = 1450
predGPA = slope*givenSAT + intercept
predGPA
```

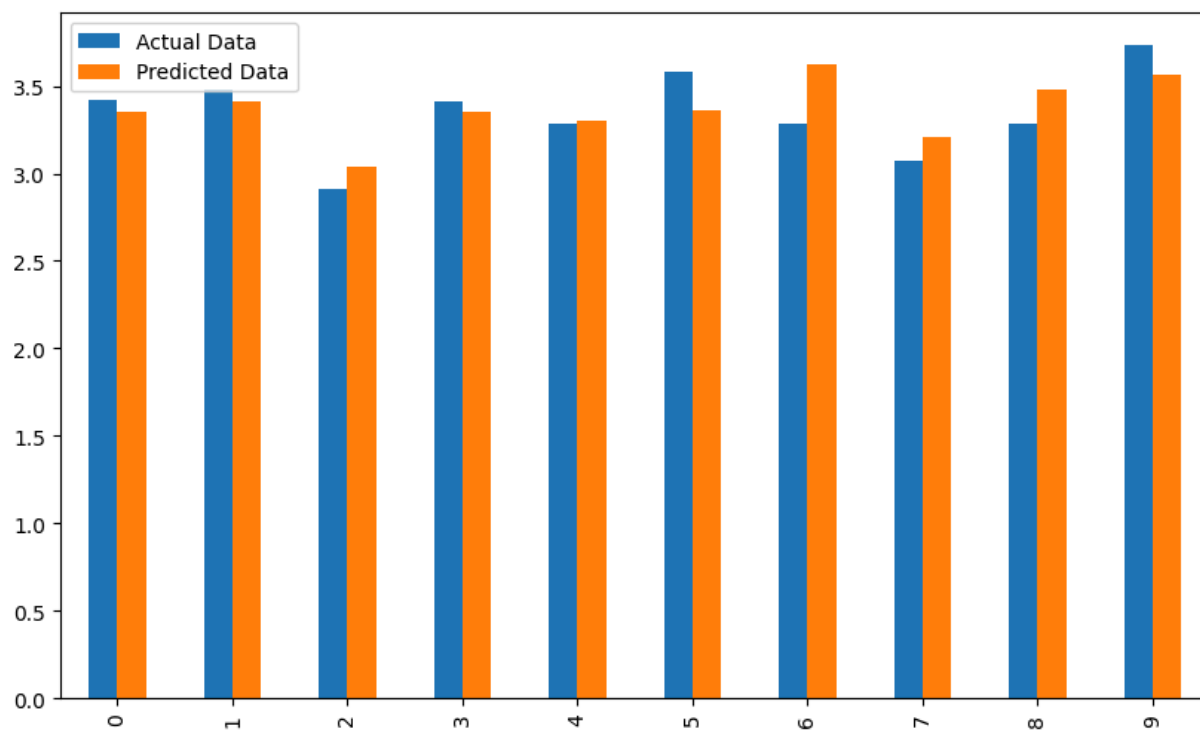
```
np.float64(3.503631997341267)
```

```
y2 = df["GPA"]
df_pred = pd.DataFrame({"Actual Data" : y2, "Predicted Data": round(y1,2)})
df_pred.head()
```

	Actual Data	Predicted Data
0	3.42	3.35
1	3.48	3.41
2	2.91	3.04
3	3.41	3.35
4	3.28	3.30

```
df_pred.head(10).plot(kind="bar", figsize=(10,6))
```

<Axes: >

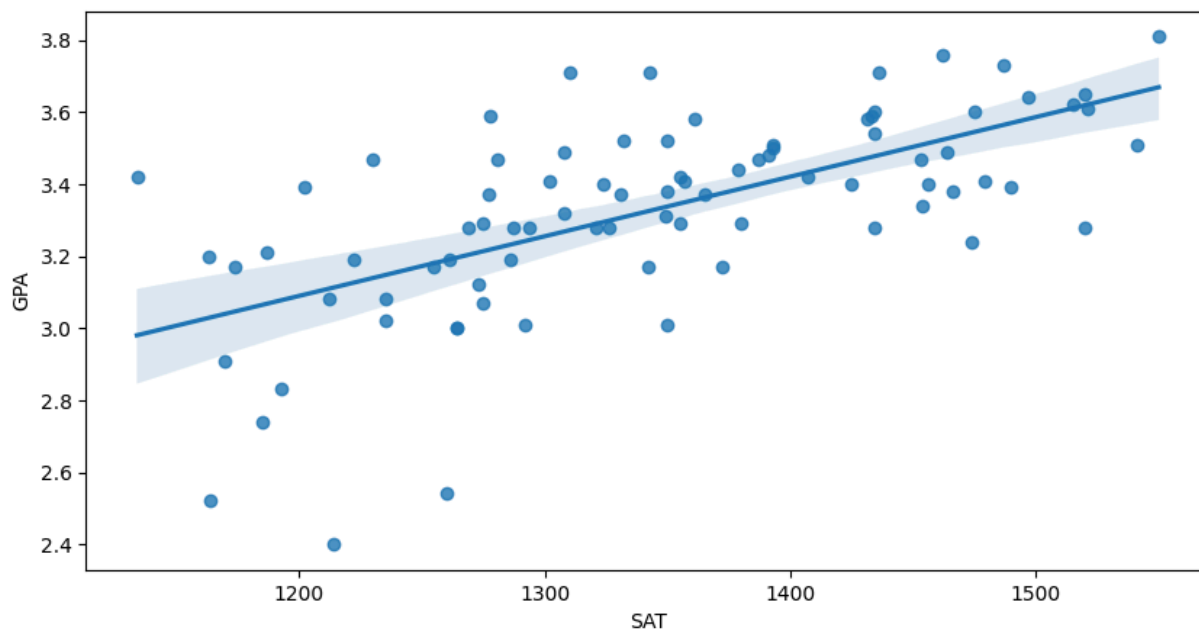


```
df_1_p = pd.DataFrame(**df, "PredictedGPA" : y1)  
df_1_p
```

	StID	SAT	GPA	PredictedGPA
0	101	1355	3.42	3.346342
1	102	1391	3.48	3.405946
2	103	1170	2.91	3.040039
3	104	1357	3.41	3.349653
4	105	1326	3.28	3.298327
...	...	...	...	...
79	193	1365	3.37	3.362899
80	195	1479	3.41	3.551647
81	196	1324	3.40	3.295015
82	197	1294	3.28	3.245345
83	198	1379	3.44	3.386078

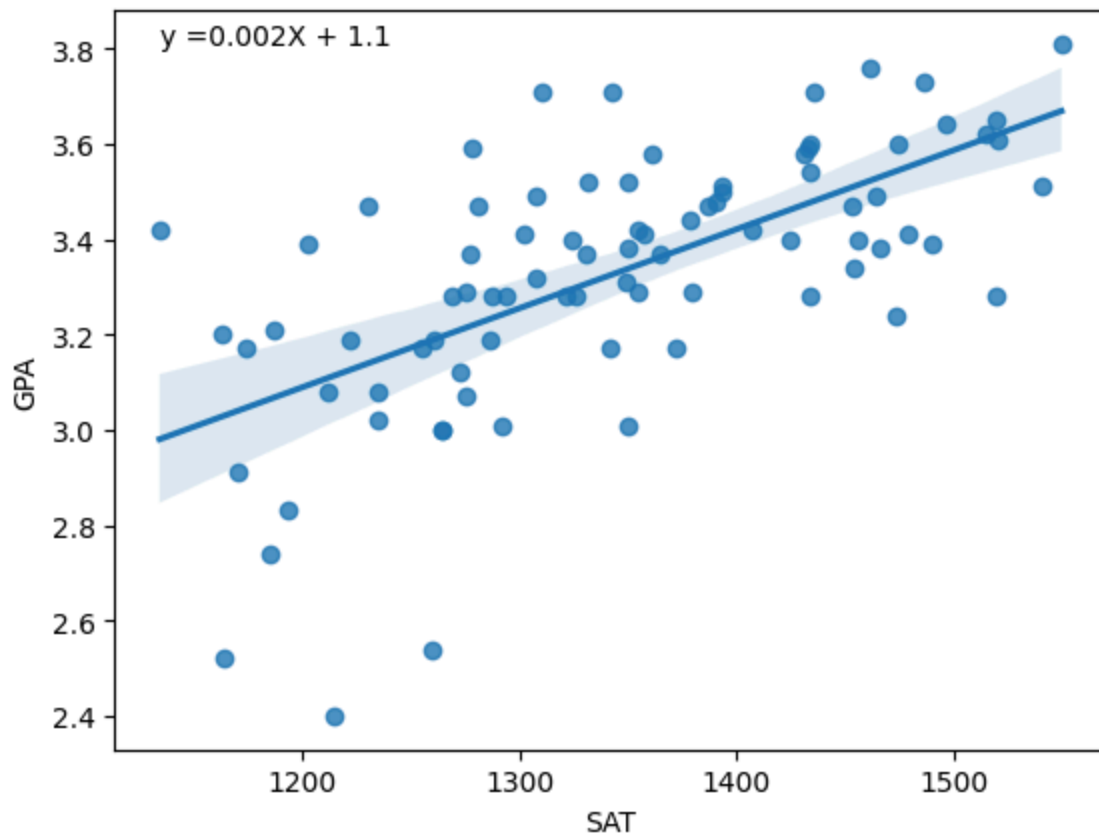
84 rows × 4 columns

```
plt.figure(figsize=(10,5))
p = sns.regplot(data=df, x=df["SAT"], y=df["GPA"])
```



```
plt.text(min(df["SAT"]), max(df["GPA"]), f"y = {round(slope, 3)}X + {round(inter\nsns.regplot(data=df, x=df[\"SAT\"], y=df[\"GPA\"])
```

<Axes: xlabel='SAT', ylabel='GPA'>



```
X = df[["SAT"]]
Y = df["GPA"]
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.3, random
```

```
clf = LinearRegression()
```

```
model = clf.fit(x_train, y_train)
```

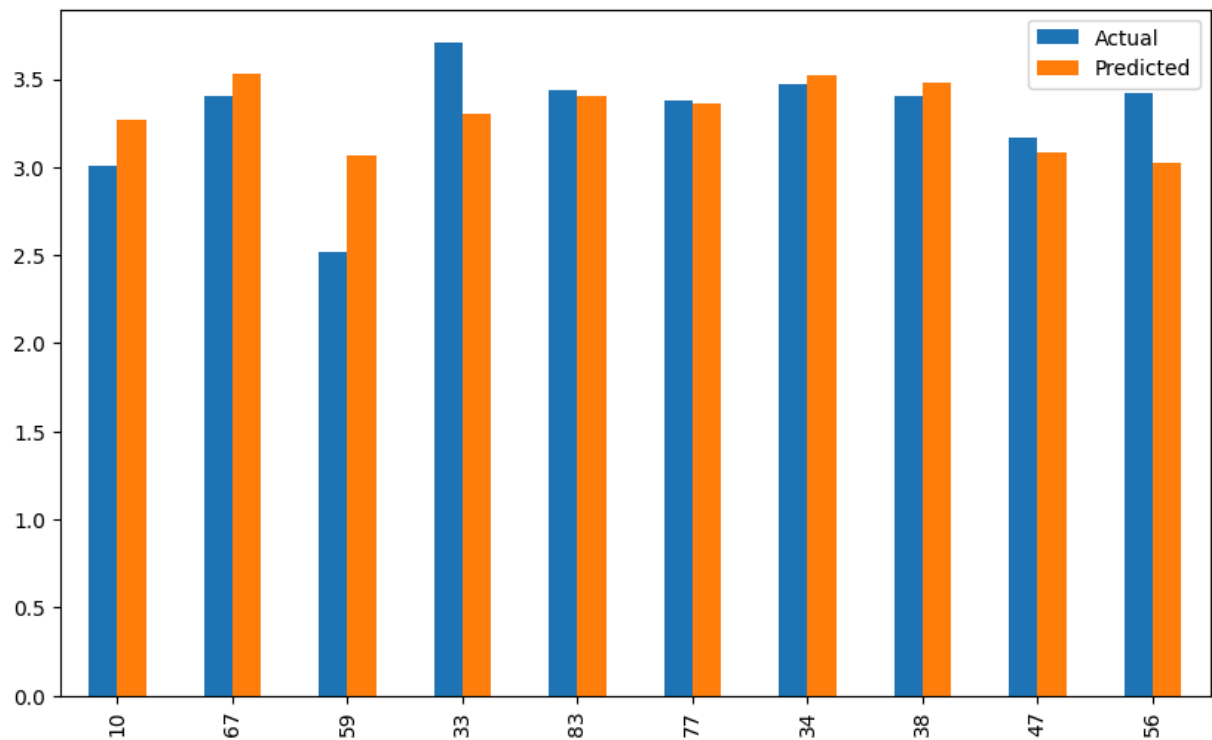
```
y_pred = model.predict(x_test)
```

```
y = pd.DataFrame({"Actual" : y_test, "Predicted" : y_pred})
y.head()
```

	Actual	Predicted
10	3.01	3.270437
67	3.40	3.527507
59	2.52	3.069796
33	3.71	3.298652
83	3.44	3.406809

```
y.head(10).plot(kind="bar", figsize=(10,6))
```

<Axes: >



```
print("MAE\t\t", metrics.mean_absolute_error(y_test, y_pred))
print("MSE\t\t", metrics.mean_squared_error(y_test, y_pred))
print("RMSE\t\t", metrics.mean_squared_error(y_test, y_pred)**0.5)
print("R_Squared\t", metrics.r2_score(y_test, y_pred))
```

```
MAE          0.2054834292112158
MSE          0.08052728657188303
```



```
RMSE 0.28377330137256224
```

```
R_squared 0.3346446677333793
```

```
model.score(x_test, y_test)
```

```
0.3346446677333793
```

If $RMSE > MSE$, There is high variation.

✓ Gradient Descent

```
b0 = 0
b1 = 0
learning_rate = 0.0000001
epochs = 1000
n = float(len(x_train))
```

```
x_vals = x_train.values.flatten()
```